

She.Dev: Lansiranje u IT

Prvi dio Full Stack modula - Frontend

Task Manager - zadaća 03

mockAPI.io

Kako bi Task Manager aplikacija radila sa stvarnim podacima, koji će se zaista negdje spremati, čuvati i ažurirati, koristit ćemo **mockAPI.io**. Ova stranica poslužit će nam da brzo i jednostavno napravimo “lažni” backend, koji se iz ugla naše Angular frontend aplikacije zapravo ponaša kao stvarni. Najprije treba kreirati novi projekat **Task Manager**, a zatim dodati dva resursa: **users** i **tasks**. Za svaki resurs treba definisati šemu po uzoru na ključeve iz pripadajućih interfejsa (*IUser* i *ITask*). Dodati dummy podatke koji će služiti za početno testiranje i uvezivanje sa frontendom. Detaljne upute dostupne u videu:

https://drive.google.com/file/d/1BufTlI6H1nIZXjpsog2RNnwmZ5QS3m70/view?usp=drive_link

TaskService i TaskApiService

Interfejs *ITask*, koji je korišten u prethodnim zadaćama, izdvojiti u zaseban dokument naziva *task.interface.ts* te ga eksportovati. Na svim mjestima gdje je dosad korišten treba ga importovati iz pomenutog dokumenta.

```
export interface ITask {  
  "id": string;  
  "title": string;  
  "projectName": string;  
  "assignedTo": string;  
  "status": string;  
  "dueDate": string;  
}
```

Kreirati dva servisa: *TaskService* i *TaskApiService*, koji će služiti za dohvaćanje i operacije nad taskovima. Servise kreirati koristeći Angular CLI (npr. `ng g s task-service --flat --skip-tests`) te u njima implementirati metode za dohvaćanje svih taskova, kreiranje novog taska, ažuriranje taska i brisanje taska. HTTP zahtjeve slati korištenjem metoda iz *HttpClient* klase te ih slati na URL iz **mockAPI.io** projekta. Ne zaboraviti u *app.config.ts* dokumentu provide-ati ovu klasu:

```
TS app.config.ts ×
src > app > TS app.config.ts > ...
You, 2 weeks ago | 1 author (You)
1  import {
2    ApplicationConfig,
3    provideBrowserGlobalErrorListeners,
4    provideZoneChangeDetection,
5  } from '@angular/core';
6  import { provideRouter } from '@angular/router';
7
8  import { routes } from './app.routes';
9  import { provideHttpClient } from '@angular/common/http';
10
11 export const appConfig: ApplicationConfig = {
12   providers: [
13     provideBrowserGlobalErrorListeners(),
14     provideZoneChangeDetection({ eventCoalescing: true }),
15     provideRouter(routes),
16     provideHttpClient(), // must be added here so HttpClient class can be used
17   ],
18 };
19
```

Umjesto dosadašnjih hardkodiranih nizova taskova u komponentama *TaskCardList* i *TaskTable* sada treba dohvatiti i prikazati stvarne podatke sa backenda. Osim toga, uvezati da:

- promjena statusa taska koja dolazi sa *TaskCard* komponente i
- brisanje taska koje dolazi iz *TaskTable* komponente

sada rezultira slanjem odgovarajućeg HTTP zahtjeva. Funkcionalnosti za kreiranje novog taska i editovanje informacija o tasku iz reda u *TaskTable* komponenti zasad nije potrebno dodavati.

UserApiService i UserService

Interfejs `IUser`, koji je korišten u prethodnoj zadaći, izdvojiti u zaseban dokument naziva `user.interface.ts` te ga eksportovati. Na svim mjestima gdje je dosad korišten treba ga importovati iz pomenutog dokumenta.

```
export interface IUser {  
  "id": string;  
  "name": string;  
  "email": string;  
  "role": string;  
  "password": string;  
}
```

Kreirati dva servisa: `UserService` i `UserApiService`, koji će služiti za dohvaćanje i operacije nad korisnicima. Servise kreirati koristeći Angular CLI te u njima implementirati metode za dohvaćanje svih korisnika, kreiranje novog korisnika te ažuriranje i brisanje postojećeg korisnika. HTTP zahtjeve slati korištenjem metoda iz `HttpClient` klase te ih slati na URL iz **mockAPI.io** projekta.

Umjesto dosadašnjeg hardkodiranog niza korisnika u `UserTable` komponenti sada treba dohvatiti i prikazati stvarne podatke sa backenda. Osim toga, uvezati da brisanje korisnika koje dolazi iz `UserTable` komponente sada rezultira slanjem odgovarajućeg HTTP zahtjeva. Funkcionalnosti za kreiranje novog korisnika i editovanje korisnika iz reda u `UserTable` komponenti zasad nije potrebno dodavati.

Napomena: Pratiti primjere servisa i API servisa sa prethodne radionice.